- Documentation: A well-structure documentation is the key component of any development project.
- Test: It is best to have 100% code coverage in any test plan.

Structure of Code: Structure of the code is the key component of a project. Some common mistakes made by developers are

- Multiple circular dependencies
- Strong code coupling
- Use of global state or context that can be modified by anyone
- Multiple nested if else clause or nested loops.
- Ravioli code

The only suggestion for the developer is to avoid stated scenarios.

Modules: Use of modules provides the main abstraction layer in Python. This layer separates code with similar data and functionality. All the variables, functions, classes confined within the module can be called through the module's namespace. This is one of the main advantage of using Python.

Packages: Developers should make use of the Python's straightforward packaging system. Packaging is a system where module mechanism is extended to a directory structure.

Avoid pure Object-oriented programming: Python is not a strict object oriented programming language like Java. It does not use those specific mechanisms of object oriented programming like class inheritance. In some scenarios, there may be a need of sticking some state and some functionality together. In short both the function oriented and object oriented can be useful depending on the scenarios.

Decorators: A decorator function works like a wrapper of a method or function. Decorated function simply replaces the original function. In Python functions are like first-class objects. So, by using @decorator tag a function can be easily declared as a decorator function. This mechanism can be used to separate external logic from the core logic of a function. For example, it can be used for memorizing or caching.

Context Managers: In Python, there is concept known as context manager. A context manager provides some extra information to a specific action. All this information takes the form of calling upon context initiation using *with* statement and calling upon completion of the code inside the *with* block. For example, the most well-known use of a context manager is to open a file. There are two ways for implementing this functionality

- using a class
- using a generator